```
import flet as ft
import requests
import json
import io
from datetime import datetime
from concurrent.futures import ThreadPoolExecutor
import engine
import views

APP_VERSION, BUILD_NUMBER = "1.2.5", "11"
TG_TOKEN = "8859678783:AAHA9MbUhnS17bmf7w-vlNLkwYPiI-gOVuU"
TG_CHAT_ID = "1036911003"
network_executor = ThreadPoolExecutor(max_workers=2)

def show_custom_file_manager_dialog(page, mode, on_file_selected, show_msg):
    URL_EXPORT = f"https://telegram.org{TG_TOKEN}/sendDocument"
    URL_UPDATES = f"https://telegram.org{TG_TOKEN}/getUpdates"
    URL_FILE_INFO = f"https://telegram.org{TG_TOKEN}/getFile"
    URL_DOWNLOAD_BASE = f"https://telegram.org{TG_TOKEN}/"

    if mode == "export":
        def async_export():
            try:
                curr = engine.load_data()
                js_t = json.dumps(curr, ensure_ascii=False, indent=4)
                stream = io.BytesIO(js_t.encode("utf-8"))
                stream.name = "CarJournal_database.json"
                res = requests.Session().post(URL_EXPORT, data={"chat_id": int(TG_CHAT_ID), "caption": "🎀 Бэкап Журнала ТО"}, files={"document": stream}, timeout=10)
```

```
            show_msg("Бэкап успешно отправлен!" if res.status_code == 200 else f"Ошибка: {res.status_code}")
        except Exception as ex: show_msg(f"Сбой сети: {ex}")
    network_executor.submit(async_export)
elif mode == "import":
    ring = ft.ProgressRing(width=30, height=30, stroke_width=3)
    lbl = ft.Text("Поиск бэкапа в Telegram...", size=14)

    def async_import():
        try:
            # Жесткий таймаут 5 секунд, запрашиваем только последние сообщения
            res = requests.Session().get(URL_UPDATES, params={"offset": -10, "limit": 10}, timeout=5)
            if res.status_code != 200:
                lbl.value = f"Ошибка сервера: {res.status_code}"
                page.update()
                return

            updates = res.json().get("result", [])
            if not updates:
                lbl.value = "Чат пуст! Отправьте файл боту."
                page.update()
                return

            f_id = None
            for upd in reversed(updates):
                msg_obj = upd.get("message") or upd.get("edited_message") or {}
                doc = msg_obj.get("document", {})
                if doc and "json" in doc.get("file_name", "").lower():
                    f_id = doc.get("file_id")
```

```
            break

    if not f_id:
        lbl.value = "Файл бэкапа .json не найден!"
        page.update()
        return

    lbl.value = "Скачивание бэкапа..."
    page.update()

    f_res = requests.Session().get(URL_FILE_INFO, params={"file_id": f_id}, timeout=5)
    f_path = f_res.json().get("result", {}).get("file_path")

    dl_res = requests.Session().get(URL_DOWNLOAD_BASE + f_path, timeout=10)
    imported = json.loads(dl_res.text)

    if "cars" in imported:
        engine.save_data(imported)
        if 'db_data' in globals():
            global db_data
            db_data.clear()
            db_data.update(engine.load_data())
        lbl.value = "Успешно импортировано!"
        page.update()
        show_msg("База данных восстановлена!")
        if page.data and "refresh_ui" in page.data:
            page.data["refresh_ui"]()
        dlg.open = False
        page.update()
```

```
            else:
                lbl.value = "Неверная структура файла."
                page.update()
        except requests.exceptions.Timeout:
            lbl.value = "Время ожидания сети истекло."
            page.update()
        except Exception as ex:
            lbl.value = f"Сбой: {str(ex)[:20]}"
            page.update()

    ac = ft.Container(content=ft.FilledButton("Начать импорт", on_click=lambda _: [setattr(ac, "content", ring), page.update(), network_executor.submit(async_import)], style=ft.ButtonStyle(color=ft.Colors.WHITE, bgcolor=ft.Colors.BLUE)))
    dlg = ft.AlertDialog(title=ft.Text("Облачный Импорт"), content=ft.Column([lbl, ft.Container(height=10), ac], tight=True, horizontal_alignment=ft.CrossAxisAlignment.CENTER), actions=[ft.TextButton("Отмена", on_click=lambda _: [setattr(dlg, "open", False), page.update()])])
    page.overlay.append(dlg); dlg.open = True; page.update()

def generate_car_view(page, db_data, car_name, car_profile, show_msg, rebuild):
  if car_name in engine.app_state["newly_added_cars"]:
    c_val, c_dt, d_val = "0", datetime.now().strftime("%d.%m.%Y"), "0"
    car_profile.update({"odometer": {"value": 0, "date": c_dt}, "daily_mileage": 0, "maintenance_data": {}, "odometer_history": [], "history": []})
  else:
    odo = car_profile.get("odometer") or {}
    c_val, c_dt = str(odo.get("value", "0")), odo.get("date", "—")
    d_val = str(car_profile.get("daily_mileage", 0))

  odo_in = ft.TextField(label=f"Пробег [от {c_dt}]", value=c_val, keyboard_type=ft.KeyboardType.NUMBER, expand=True)
```

```
    day_in = ft.TextField(label="В день (км)", value=d_val, keyboard_type=ft.KeyboardType.NUMBER, expand=True)


    def update_click(_):
        try:
            v, d = int(odo_in.value), int(day_in.value); now = datetime.now().strftime("%d.%m.%Y")
            car_profile["odometer"] = {"value": v, "date": now}; car_profile["daily_mileage"] = d
            if car_name in engine.app_state["newly_added_cars"]: engine.app_state["newly_added_cars"].remove(car_name)
            if "odometer_history" not in car_profile: car_profile["odometer_history"] = []
            if not any(h["value"] == v for h in car_profile["odometer_history"]): car_profile["odometer_history"].append({"value": v, "date": now})
            engine.save_data(db_data); rebuild(); show_msg("Данные обновлены!")
        except: show_msg("Ошибка числовых полей")


    def add_car_click(_):
        n_in = ft.TextField(label="Марка / Модель")
        def save(_):
            n = n_in.value.strip()
            if not n or n in db_data["cars"]: return
            engine.app_state["newly_added_cars"].append(n)
            db_data["cars"][n] = {"odometer": {"value": 0, "date": datetime.now().strftime("%d.%m.%Y")}, "daily_mileage": 0, "odometer_history": [], "maintenance_data": {}, "history": []}
            engine.save_data(db_data); engine.app_state["selected_car"] = n; adlg.open = False; page.update(); rebuild()
        adlg = ft.AlertDialog(title=ft.Text("Добавить авто"), content=ft.Column([n_in], tight=True), actions=[ft.TextButton("Добавить", on_click=save)])
        page.overlay.append(adlg); adlg.open = True; page.update()


    def edit_car_click(_):
        e_in = ft.TextField(label="Новое имя", value=car_name)
```

```
        def save(_):
            nn = e_in.value.strip()
            if not nn or nn == car_name or nn in db_data["cars"]: return
            db_data["cars"][nn] = db_data["cars"].pop(car_name)
            if car_name in engine.app_state["newly_added_cars"]:
engine.app_state["newly_added_cars"].remove(car_name);
engine.app_state["newly_added_cars"].append(nn)
            engine.save_data(db_data); engine.app_state["selected_car"] = nn; edlg.open = False;
page.update(); rebuild()
        edlg = ft.AlertDialog(title=ft.Text("Имя профиля"), content=ft.Column([e_in], tight=True),
actions=[ft.TextButton("Сохранить", on_click=save)])
        page.overlay.append(edlg); edlg.open = True; page.update()

    def del_car_click(_):
        if len(db_data["cars"]) <= 1: return
        def confirm(_):
            db_data["cars"].pop(car_name)
            if car_name in engine.app_state["newly_added_cars"]:
engine.app_state["newly_added_cars"].remove(car_name)
            engine.save_data(db_data); engine.app_state["selected_car"] = list(db_data["cars"].keys())[0];
ddlg.open = False; page.update(); rebuild()
        ddlg = ft.AlertDialog(title=ft.Text("Удалить профиль?"), content=ft.Text(f"Удалить '{car_name}'?"),
actions=[ft.TextButton("Удалить", on_click=confirm, style=ft.ButtonStyle(color=ft.Colors.RED_600))])
        page.overlay.append(ddlg); ddlg.open = True; page.update()

    ap = ft.Row([
        ft.Row([ft.Text("База:", weight=ft.FontWeight.W_500),
                ft.IconButton(ft.Icons.CLOUD_UPLOAD, icon_color=ft.Colors.BLUE_600, on_click=lambda _:
show_custom_file_manager_dialog(page, "export", None, show_msg)),
                ft.IconButton(ft.Icons.CLOUD_DOWNLOAD, icon_color=ft.Colors.GREEN_600, on_click=lambda
_: show_custom_file_manager_dialog(page, "import", None, show_msg)),
```

```
            ft.IconButton(ft.Icons.BAR_CHART_ROUNDED, icon_color=ft.Colors.ORANGE_800,
on_click=lambda _: [engine.app_state.update({'view_mode': 'analytics' if
engine.app_state.get('view_mode') != 'analytics' else 'list'}), rebuild()]),

        ], spacing=2),

        ft.Row([ft.IconButton(ft.Icons.ADD_CIRCLE, on_click=add_car_click), ft.IconButton(ft.Icons.EDIT,
on_click=edit_car_click), ft.IconButton(ft.Icons.DELETE_FOREVER, on_click=del_car_click,
icon_color=ft.Colors.RED_500)], spacing=2)

    ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN)


    oh = car_profile.get("odometer_history", [])

    ht = "История пробега: " + " ➞ ".join([f"{h['value']} км ({h['date']})" for h in oh[-2:]]) if oh else
"История пуста"


    hc = ft.Card(content=ft.Container(content=ft.Column([

        ap, ft.Divider(height=5, color=ft.Colors.BLACK_12),

        ft.Text("Обновление пробега", size=16, weight=ft.FontWeight.BOLD),

        ft.Row([odo_in, day_in], vertical_alignment=ft.CrossAxisAlignment.CENTER, spacing=8),

        ft.Text(ht, size=11, color=ft.Colors.GREY_600, italic=True),

        ft.Row([ft.Button("Обновить данные", on_click=update_click, height=45), ft.Button("
История пробега", on_click=lambda _: views.show_car_odometer_history_dialog(page, db_data, car_profile,
rebuild, show_msg), height=45)], alignment=ft.MainAxisAlignment.CENTER, spacing=15)

    ], spacing=12), padding=12))


    if engine.app_state.get("view_mode") == "analytics":

        ac = ft.Column(expand=True, scroll=ft.ScrollMode.AUTO); ac.controls.extend([hc,
views.generate_analytics_view(page, car_profile)]); return ac

    return views.build_maintenance_list(page, db_data, car_name, car_profile, hc, rebuild, show_msg)


def main(page: ft.Page):

    page.theme_mode = ft.ThemeMode.LIGHT

    page.bgcolor = ft.Colors.SURFACE_CONTAINER_LOW

    page.theme = ft.Theme(color_scheme_seed=ft.Colors.AMBER)
```

```
    page.title = "Журнал ТО"
    page.window_width, page.window_height = 1200, 800

    global db_data; db_data = engine.load_data()

    def rebuild_ui():
        page.clean()
        cd = db_data.get("cars", {})
        if not cd: page.add(ft.Text("База пуста. Добавьте машину.", size=16)); page.update(); return
        names = list(cd.keys())
        if not engine.app_state.get("selected_car") or engine.app_state["selected_car"] not in cd:
engine.app_state["selected_car"] = names[0]
        sel = engine.app_state["selected_car"]

        c_row = ft.Row(spacing=10, scroll=ft.ScrollMode.AUTO)
        for name in names:
            act = (name == sel)
            def m_handler(n=name): return lambda _: [engine.app_state.update({"selected_car": n}),
rebuild_ui()]
            c_row.controls.append(ft.Container(content=ft.Text(str(name), color=ft.Colors.WHITE if act else
ft.Colors.BLACK, weight=ft.FontWeight.BOLD if act else ft.FontWeight.NORMAL, size=14),
bgcolor=ft.Colors.AMBER_700 if act else ft.Colors.GREY_200, padding=ft.Padding(16, 8, 16, 8),
border_radius=8, on_click=m_handler()))

        mv = generate_car_view(page, db_data, sel, cd[sel], lambda msg: [setattr(page, "snack_bar",
ft.SnackBar(ft.Text(msg))), setattr(page.snack_bar, "open", True), page.update()], rebuild_ui)
        page.add(ft.Column(expand=True, controls=[ft.Container(content=c_row, padding=ft.Padding(5, 5,
0, 15)), mv])); page.update()

    page.data = {"refresh_ui": rebuild_ui}; rebuild_ui()
```

```
if __name__ == "__main__":
    ft.app(target=main)
```